



Linguaggi di descrizione dell'hardware

Il progetto nella pratica di oggi

Preview

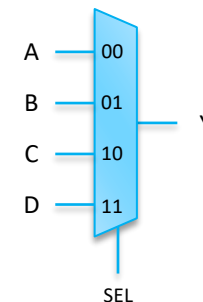
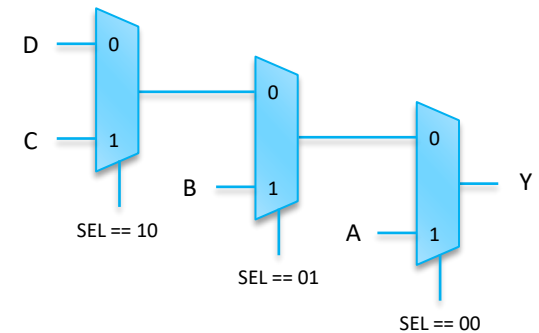
► Come funziona?

- Scriviamo del software che viene poi trasformato in un circuito
- Cerchiamo di usare concetti ad alto livello

► Esempi

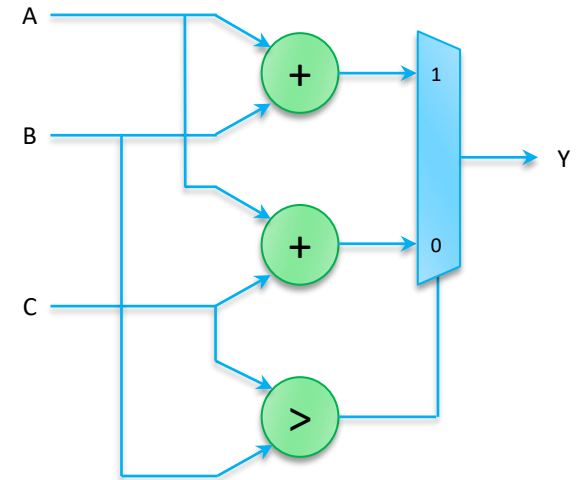
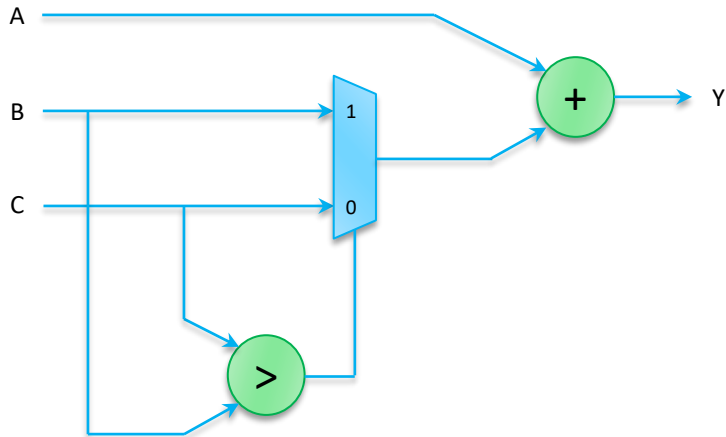
```
if      (sel == 00) then Y = A;  
elseif (sel == 01) then Y = B;  
elseif (sel == 10) then Y = C;  
else      Y = D;
```

```
case (sel) {  
  00: Y = A; break;  
  01: Y = B; break;  
  10: Y = C; break;  
  11: Y = D; break;  
}
```



Preview, operazione complessa

```
if (B > C) then  
    Y = A + B;  
else  
    Y = A + C;
```



II VHDL

Lo prendiamo come esempio

Il linguaggio VHDL

- ▶ **Very High Speed Integrated Circuit Hardware Description Language**
 - ▶ VHSIC HDL = VHDL
- ▶ **Scopi**
 - ▶ Documentazione, simulazione, sintesi
- ▶ **Metodologie**
 - ▶ Top-down – procede da un livello alto ad uno basso scomponendo un sistema in sottosistemi
 - ▶ Bottom-up – ottiene sistemi complessi assemblando sistemi più semplici
 - ▶ Meet-in-the-middle – decompone un sistema in sottosistemi, fino ad arrivare ad elementi contenuti in una libreria
- ▶ **Viste**
 - ▶ Data flow
 - ▶ Strutturale
 - ▶ Comportamentale

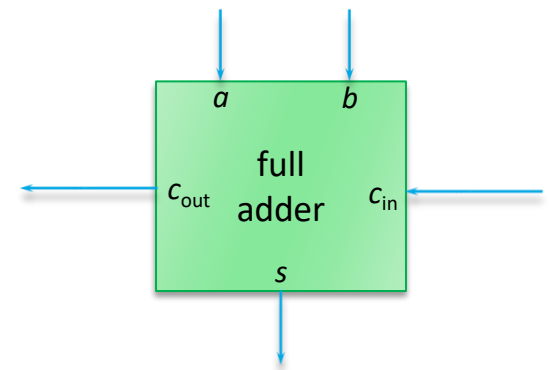
Concetti base: entità

► L'entità e l'oggetto base del VHDL

- Corrisponde ad un blocco, un modulo, un elemento
- Simile ad una classe in C++, o una funzione in C
- E' identificata da un nome
- Dispone di un certo numero di ingressi ed uscite
- **Definisce l'interfaccia del blocco**

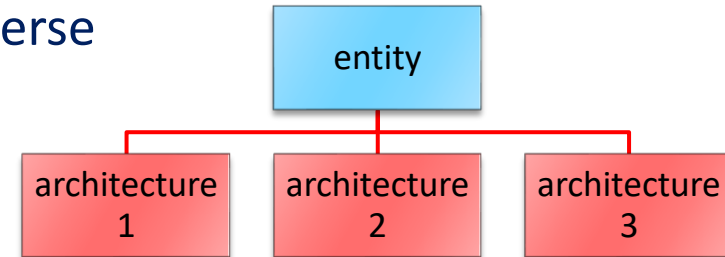
```
entity entity_name is  
  [port (interface-signal-declaration);]  
end [entity] [entity_name];
```

```
entity full_adder is  
  port (a, b, cin : in bit;  
        s, cout : out bit);  
end full_adder;
```



Concetti base: architettura

- ▶ **L'architettura definisce cosa fa un blocco**
 - ▶ Ogni architettura corrisponde ad una entità
 - ▶ Ogni entità può avere più architetture diverse
 - ▶ **Ne descrive il funzionamento** tramite equazioni, connessioni strutturali di altri blocchi (gerarchia), oppure tramite un algoritmo



```
architecture architecture_name of entity_name is  
    [declarations]  
begin  
    [architecture body]  
end [architecture] [architecture_name];
```

```
architecture componenti of full_adder is  
begin  
    blah blah blah  
end componenti;
```

Stili di descrizione

▶ **Data flow o equazioni**

- ▶ Descrive il funzionamento tramite delle equazioni booleane
- ▶ Esprime la dipendenza delle uscite in funzione degli ingressi e di segnali interni
- ▶ Le varie equazioni sono messe a sistema

▶ **Strutturale**

- ▶ Descrive il funzionamento come collegamento di componenti in una gerarchia
- ▶ La nuova entità può essere usata a sua volta come componente in un'altra architettura

▶ **Behavioral o comportamentale**

- ▶ Descrive il funzionamento tramite un algoritmo
- ▶ Il codice descrive come derivare il valore dell'uscita in funzione di quello degli ingressi, dei segnali interni e di variabili

Esempio di porta AND a 2 ingressi

► Interfaccia

- Ingressi x e y , uscita z

► Comportamento

- Usiamo l'operatore **and** già predefinito nel linguaggio
- L'operatore **<=** esegue l'assegnazione

Nomi dei segnali di interfaccia

in denota i segnali di ingresso

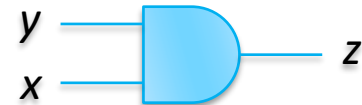
out denota i segnali di uscita

bit indica un tipo di segnali che può avere valori 0 e 1

Equazione che descrive il comportamento del componente

```
-- Porta AND a 2 ingressi
entity And2 is
  port (x, y : in bit; z : out bit);
end entity And2

architecture ex1 of And2 is
begin
  z <= x and y;
end architecture ex1;
```



Testbench

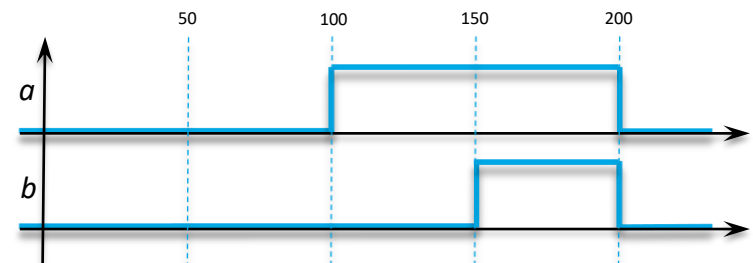
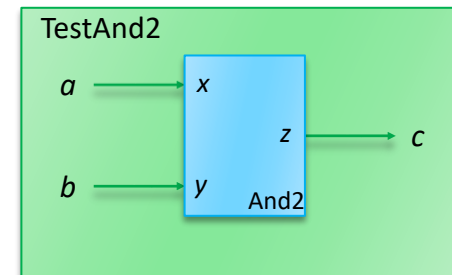
- ▶ Per poter eseguire una simulazione occorre fornire dei valori ai segnali di ingresso
 - ▶ Il blocco che realizza questa funzione è solitamente chiamato testbench
 - ▶ E' possibile realizzarlo mischiando la rappresentazione data flow con quella strutturale
 - ▶ Il testbench chiude il sistema, quindi l'entità non ha ingressi e uscite

```
-- Porta AND a 2 ingressi
entity And2 is
  port (x, y : in bit; z : out bit);
end entity And2;
```

```
architecture ex1 of And2 is
begin
  z <= x and y;
end architecture ex1;
```

```
-- Testbench per porta And a 2 ingressi
entity TestAnd2 is
end entity TestAnd2;

architecture simple of TestAnd2 is
  -- Segnali interni di interconnessione
  signal a, b, c : bit;
begin
  -- Istanza del modulo da testare
  g1: And2 port map (x => a, y => b, z => c);
  -- Definizione degli stimoli
  a <= '0', '1' after 100 ns, '0' after 200 ns;
  b <= '0', '1' after 150 ns, '0' after 200 ns;
end architecture simple;
```



Operatori e identificatori

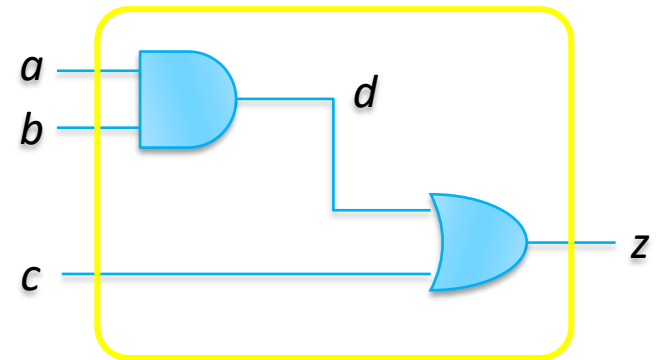
- ▶ Il VHDL dispone di tutti gli operatori dell'algebra booleana
 - ▶ not
 - ▶ and, or
 - ▶ nand, nor
 - ▶ xor, xnor
- ▶ **Precedenza degli operatori**
 - ▶ not ha la precedenza sugli altri
 - ▶ Gli altri hanno tutti la stessa precedenza, applicati da sinistra a destra
 - ▶ Quindi attenzione a mettere bene le parentesi!!
- ▶ **Identificatori**
 - ▶ Utilizzati per nomi di entità, architetture, segnali
 - ▶ Il VHDL è case insensitive, quindi x e X sono la stessa cosa!

Espressioni multiple

- ▶ Possiamo usare diverse espressioni legate da segnali interni
 - ▶ Dichiariamo il nome e il tipo dei segnali interni prima delle equazioni
 - ▶ L'ordine delle equazioni non ha importanza

```
-- Uso di espressioni multiple
entity comb_function is
  port (a, b, c : in bit; z : out bit);
end entity comb_function;

architecture expression of comb_function is
  signal d : bit;
begin
  d <= a and b;
  z <= d or c;
end architecture expression;
```

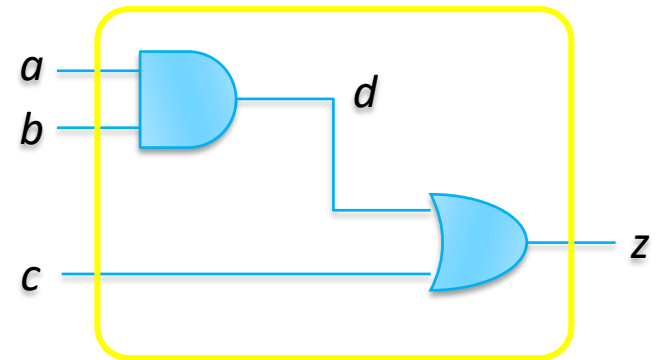


Ritardi

- ▶ **Si può specificare il ritardo di una espressione**
 - ▶ Si usa l'operatore **after** seguito dall'indicazione del tempo
 - ▶ Il risultato non viene assegnato al segnale destinazione fin quando non è passato il tempo di simulazione
 - ▶ Se non viene specificato alcun ritardo si assume comunque un ritardo arbitrariamente piccolo indicato con δ

```
-- Uso di espressioni multiple
entity comb_function is
  port (a, b, c : in bit; z : out bit);
end entity comb_function;

architecture expression of comb_function is
  signal d : bit;
begin
  z <= d or c after 4 ns;
  d <= a and b after 5 ns;
end architecture expression;
```



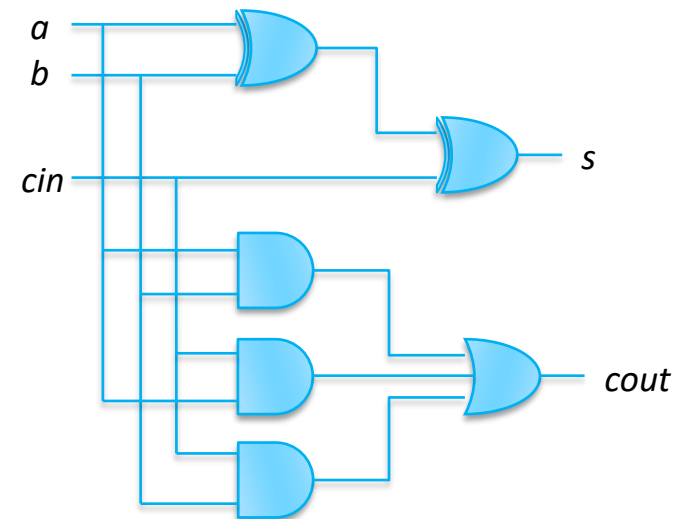
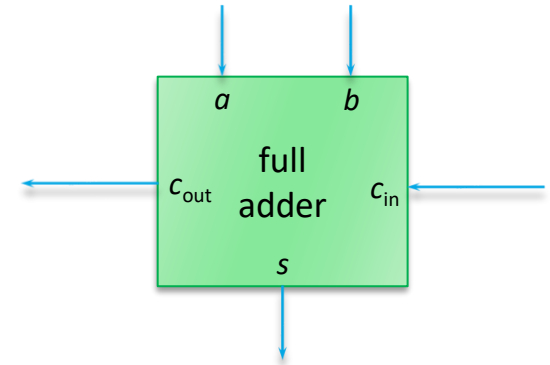
VHDL in pratica

Vista strutturale, sintassi, procedurale, etc.

II full adder

```
entity full_adder is
  port (a, b, cin : in bit;
        s, cout : out bit);
end full_adder;
```

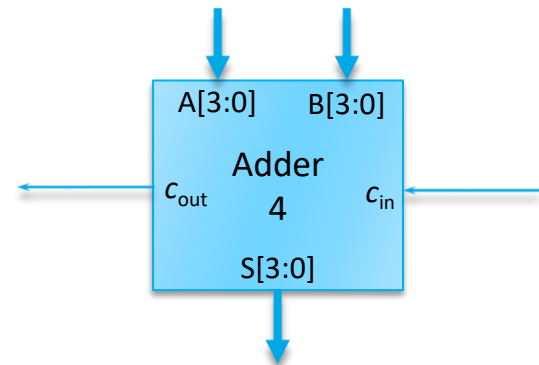
```
architecture equazioni of full_adder is
begin
  s <= a xor b xor cin;
  cout <= (a and b) or (a and cin)
          or (b and cin);
end architecture equazioni;
```



Sommatore a 4 bit

- ▶ Dobbiamo poter definire un'interfaccia con segnali multibit
- ▶ Il VHDL supporta la definizione di array di valori, usando il tipo `bit_vector`
- ▶ Possiamo definire un range qualunque

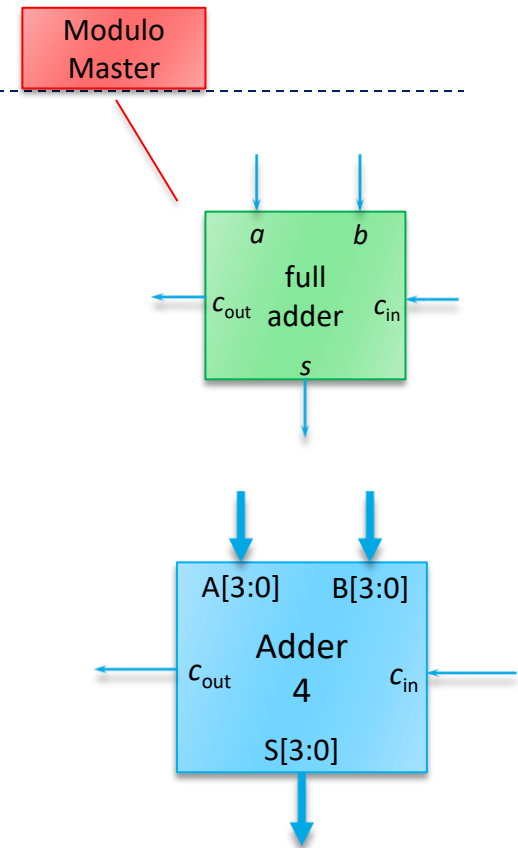
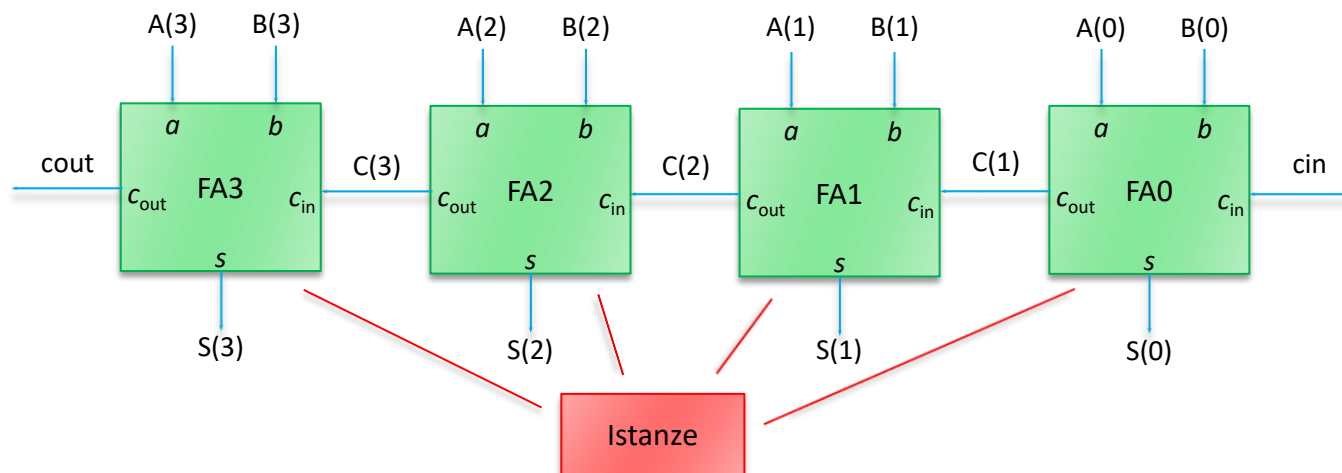
```
entity Adder4 is
  port (A, B : in bit_vector(3 downto 0);
        cin : in bit;
        S : out bit_vector(3 downto 0);
        cout : out bit);
end Adder4;
```



Architettura strutturale

► Costruiamo l'addizionatore a 4 bit mettendo assieme 4 full adder

- Nell'architettura si specifica il modulo che si vuole istanziare e si dà un nome ad ogni istanza
 - Per esempio FA0, FA1, FA2 ed FA3
- Quindi si definisce una mappatura tra i segnali del modulo e quelli del composito
 - I segnali A(0) e B(0) andranno ad FA0
 - I segnali A(1) e B(1) andranno ad FA1, etc.



Parametri formali ed effettivi

- ▶ **Ogni componente assegna un nome agli ingressi e uscite**

- ▶ In termini informatici, questo è chiamato *parametro formale*
- ▶ Tale nome ha significato all'interno del componente

```
entity full_adder is
  port (a, b, cin: in bit;
        cout, s: out bit);
end full_adder;
```

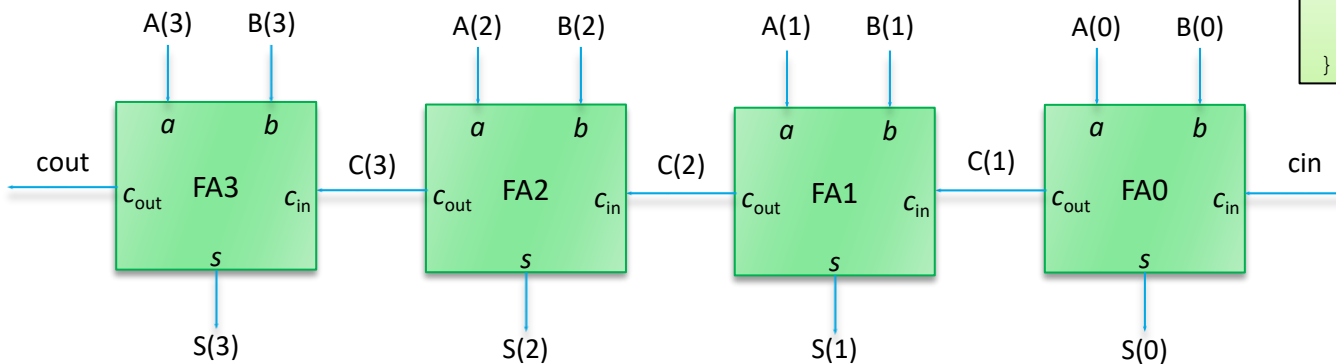
- ▶ **Quando si usa il componente, si assegna ad ogni parametro formale un valore effettivo**

- ▶ Si traduce la porta di ingresso/uscita nel contesto del livello di gerarchia superiore
- ▶ Per esempio, A(0) è collegato alla porta *a* di FA0
- ▶ Mentre A(1) è collegato alla porta *a* di FA1
- ▶ La porta *cout* di FA0 è collegata a C(1), al quale è collegata anche la porta *cin* di FA1

- ▶ **E' come quando si chiamano le funzioni in C**

- ▶ Si passano valori del contesto chiamante, che sono usati in quello chiamato

```
int f( int primo, int secondo ) {
  return primo + secondo;
}
int main( void ) {
  int a = 3;
  int b = 5;
  return f( a, b );
}
```



Architettura strutturale in VHDL

► Segue la sintassi di chiamata a funzione

- Si indica nome dell'istanza e del modulo da istanziare
- Si indicano in ordine le porte che si vogliono collegare tramite un port map
- Notazione posizionale come in C

```
entity Adder4 is
  port (A, B : in bit_vector(3 downto 0);
        cin : in bit;
        S : out bit_vector(3 downto 0);
        cout : out bit);
end Adder4;
```

```
architecture strutturale of Adder4 is
  signal C : bit_vector(3 downto 1);
begin
  FA0: full_adder port map (A(0), B(0), cin, C(1), S(0));
  FA1: full_adder port map (A(1), B(1), C(1), C(2), S(1));
  FA2: full_adder port map (A(2), B(2), C(2), C(3), S(2));
  FA3: full_adder port map (A(3), B(3), C(3), cout, S(3));
end architecture strutturale;
```

```
entity full_adder is
  port (a, b, cin: in bit;
        cout, s: out bit);
end full_adder;
```

Nome dell'istanza

Nome del master

Corrispondenza porte

Associazione esplicita

- ▶ Invece di usare una notazione posizionale è possibile indicare esplicitamente le connessioni
 - ▶ C'è più roba da scrivere
 - ▶ Non importa l'ordine
 - ▶ Più comodo quando il file VHDL viene generato da un tool

```
entity Adder4 is
  port (A, B : in bit_vector(3 downto 0);
        cin : in bit;
        S : out bit_vector(3 downto 0);
        cout : out bit);
end Adder4;
```

```
entity full_adder is
  port (a, b, cin: in bit;
        s, cout: out bit);
end full_adder;
```

Stesso nome ma oggetti diversi

```
architecture strutturale of Adder4 is
  signal C : bit_vector(3 downto 1);
begin
  FA0: full_adder port map (a => A(0), b => B(0), cin => cin, cout => C(1), s => S(0));
  FA1: full_adder port map (b => B(1), a => A(1), cout => C(2), cin => C(1), s => S(1));
  FA2: full_adder port map (cout => C(3), s => S(2), a => A(2), b => B(2), cin => C(2));
  FA3: full_adder port map (cin => C(3), a => A(3), b => B(3), cout => cout, s => S(3));
end architecture strutturale;
```

Modi dei segnali

► Segnali di ingresso (in)

- Possono essere soltanto letti
- Nelle equazioni possono comparire solo a destra del segno <=

► Segnali di uscita (out)

- Possono essere soltanto scritti
- Nelle equazioni possono comparire solo a sinistra del segno <=
- Devono essere definiti da una sola espressione, altrimenti ci potrebbero essere dei conflitti

```
entity full_adder is
  port (a, b, cin : in bit; s, cout : out bit);
end full_adder;
architecture equazioni of full_adder is begin
  s <= a xor b xor cin;
  cout <= (a and b) or (a and cin) or (b and cin);
  a <= b xnor cin;
  s <= a and b;
  cout <= a or s;
end architecture equazioni;
```

Ingresso appare a sinistra

Equazioni di uscita in conflitto

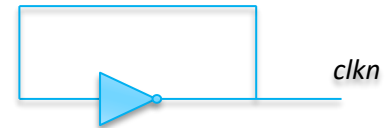
Uscita appare a destra

Modi dei segnali

- ▶ **Segnali di uscita che possono essere anche letti (buffer)**
 - ▶ Devono essere definiti come **buffer** nella definizione di entità
 - ▶ Possono comparire da entrambi i lati del segno `<=`
 - ▶ Essendo uscite, devono comparire a sinistra di `<=` in una sola espressione

```
-- Generatore di clock a 100 MHz
entity clock_generator is
  port (clk_n : buffer bit);
end clock_generator;

architecture equazioni of clock_generator is
begin
  clk_n <= not clk_n after 5 ns;
end architecture equazioni;
```



Modi dei segnali

► Segnali interni dell'architettura (signal)

- Possono essere sia letti che scritti
- Si comportano come i **buffer**
- Comunque devono essere scritti una volta sola
- In particolare, quando sono parte di istanziazioni, devono essere scritti una volta sola quando si espandono le istanziazioni con le equazioni
- Quindi attenzione a come si collegano i componenti (non collegare le uscite di due componenti assieme!)

```
architecture equazioni of Adder4 is
    signal C : bit_vector(3 downto 1);
begin
    S(0) <= A(0) xor B(0) xor cin;
    C(1) <= (A(0) and B(0)) or (A(0) and cin) or (B(0) and cin);
    S(1) <= A(1) xor B(1) xor C(1);
    C(2) <= (A(1) and B(1)) or (A(1) and C(1)) or (B(1) and C(1));
    S(2) <= A(2) xor B(2) xor C(2);
    C(3) <= (A(2) and B(2)) or (A(2) and C(2)) or (B(2) and C(2));
    S(3) <= A(3) xor B(3) xor C(3);
    cout <= (A(3) and B(3)) or (A(3) and C(3)) or (B(3) and C(3));
end architecture equazioni;
```

```

architecture equazioni of full_adder is
begin
    s <= a xor b xor cin;
    cout <= (a and b) or (a and cin)
           or (b and cin);
end architecture equazioni;

```

```

architecture strutturale of Adder4 is
    signal C : bit_vector(3 downto 1);
begin
    FA0: full_adder port map (A(0), B(0), cin, C(1), S(0));
    FA1: full_adder port map (A(1), B(1), C(1), C(2), S(0));
    FA2: full_adder port map (A(2), B(2), C(2), C(3), S(2));
    FA3: full_adder port map (A(3), B(3), C(3), cout, S(3));
end architecture strutturale;

```

```

architecture strutturale of Adder4 is
    signal C : bit_vector(3 downto 1);
begin
    S(0) <= A(0) xor B(0) xor cin;
    C(1) <= (A(0) and B(0)) or (A(0) and cin) or (B(0) and cin);
    S(0) <= A(1) xor B(1) xor C(1);
    C(2) <= (A(1) and B(1)) or (A(1) and C(1)) or (B(1) and C(1));
    S(2) <= A(2) xor B(2) xor C(2);
    C(3) <= (A(2) and B(2)) or (A(2) and C(2)) or (B(2) and C(2));
    S(3) <= A(3) xor B(3) xor C(3);
    cout <= (A(3) and B(3)) or (A(3) and C(3)) or (B(3) and C(3));
end architecture strutturale;

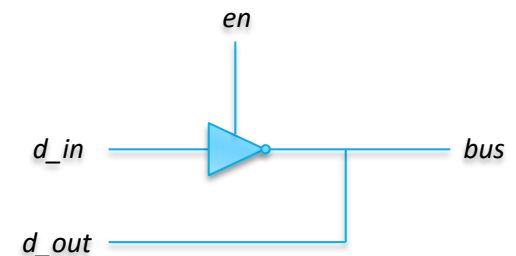
```

Modi dei segnali

► Segnali sia di ingresso che di uscita (inout)

- Dal punto di vista del componente sono come i **buffer**
- Ma possono avere un driver anche dall'esterno, mentre i **buffer** no
- Possono apparire a sinistra di $\leq$ in più espressioni
- Occorre però risolvere eventuali conflitti
- Usati per lo più per rappresentare buffer tri-state
- L'uso effettivo richiede un ulteriore tipo di dato (a breve!)

```
-- Interfaccia tri-state verso il bus
entity bus_interface is
  port (en, d_in : in bit;
        d_out : out bit
        bus : inout bit);
end entity bus_interface;
```



Parametri (generic)

- ▶ **E' spesso utile poter parametrizzare un componente**
 - ▶ In questo modo ne scriviamo la descrizione una sola volta
 - ▶ Quando lo si istanzia si definisce il valore del parametro

```
-- Porta con ritardo parametrizzato
entity And2 is
  generic (delay : time);
  port (x, y : in bit; z : out bit);
end entity And2;
architecture ex2 of And2 is begin
  z <= x and y after delay;
end architecture ex2;

...
g1: And2 generic map (5 ns)
  port map (p, b, q);
g2: And2 generic map (delay => 3 ns)
  port map (z => r, y => p, x => s);
...
```

Definizione parametro

Uso del parametro

Istanziamento del
parametro

Parametri (generic)

► E' possibile fornire un valore di default

- Il valore di default viene usato quando non viene specificato nulla nell'istanza
- Si può comunque modificarne il valore come al solito

```
-- Porta con ritardo parametrizzato e default
entity And2 is
  generic (delay : time := 5 ns);
  port (x, y : in bit; z : out bit);
end entity And2;
architecture ex2 of And2 is begin
  z <= x and y after delay;
end architecture ex2;

...
g1: And2 port map (p, b, q);

g2: And2 generic map (delay => 3 ns)
  port map (z => r, y => p, x => s);
...
```

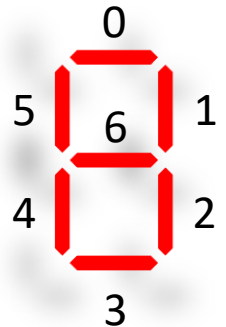
Definizione parametro
con valore di default

Istanziamento con uso
del valore di default

Costanti

- ▶ **Assieme ai segnali è possibile definire delle costanti**
 - ▶ **constant** nome_costante: tipo := valore
 - ▶ Utili per definire valori simbolici da poter usare nel resto del codice
 - ▶ Per esempio per definire le dimensioni degli array, o dare valori simbolici alla codifica degli stati
 - ▶ Se occorre modificarli, basta farlo in un punto solo del codice

```
-- Uso di costanti
architecture equazioni of display is
    constant segmenti : integer := 7;
    constant zero : bit_vector(segmenti-1 downto 0) := "0111111";
    constant uno : bit_vector(segmenti-1 downto 0) := "0000110";
    ...
    constant cinque : bit_vector(segmenti-1 downto 0) := "1101101";
    ...
begin
    uscita <= cinque when (ingresso = 5);
end architecture equazioni;
```



Tipi

▶ Ogni segnale deve avere un tipo

- ▶ Ne caratterizza i possibili valori
- ▶ Abbiamo visto il tipo `bit` e `bit_vector`
 - ▶ I segnali di questo tipo possono assumere i valori '0' oppure '1'
- ▶ Con la keyword **downto** si definisce l'ordinamento dei bit
 - ▶ `b : in bit_vector (3 downto 0);`
 - ▶ `b <= "1100"` assegna a `b(3)` e `b(2)` il valore 1, e a `b(1)` e `b(0)` il valore 0
- ▶ La keyword **to** definisce l'ordinamento contrario
 - ▶ `b : in bit_vector (3 to 0);`
 - ▶ `b <= "1100"` assegna a `b(3)` e `b(2)` il valore 0, e a `b(1)` e `b(0)` il valore 1

▶ Altri tipi

- ▶ Abbiamo visto il tipo "integer"
- ▶ Abbiamo visto il tipo "time"

Tipi definiti dall'utente

- ▶ **Possiamo definire i tipi che vogliamo**
 - ▶ Come il typedef in C
 - ▶ Aumenta la leggibilità del codice e permette di evitare errori
 - ▶ **type word is bit_vector (15 downto 0);**

```
-- Uso di tipi definiti dall'utente
architecture equazioni of display is
    constant segmenti : integer := 7;
    type 7segments is bit_vector(segmenti-1 downto 0);
    constant zero : 7segments := "0111111";
    constant uno  : 7segments := "0000110";
    ...
begin
    uscita <= cinque and (ingresso = 5);
end architecture equazioni;
```

Tipi compositi

- ▶ **Il VHDL offre i normali modi di definire tipi compositi**
 - ▶ Consentono di rendere il codice più simbolico e meno dipendente dalla specifica implementazione

```
-- Tipi enumerati
type op_select_type is (load, store, add, sub, div, mult, shiftl, shiftr);
type hex_digit is ('0', '1', '2', '3', '4', '5', '6', '7', '8',
                  '9', 'A', 'B', 'C', 'D', 'E', 'F');
type state_type is (S0, S1, S2, S3);

-- Tipi array
type my_word is array (15 downto 0) of hex_digit;

-- Tipi struttura
type packet is record
    RISE_TIME : time;
    FALL_TIME : time;
    SIZE : integer range 0 to 200;
    DATA : bit_vector (15 downto 0);
end record;
...
signal A, B : packet;
...
A.RISE_TIME <= 5ns;
A.SIZE <= 120;
B <= A;
```

Il tipo std_logic

- ▶ **Definito nella libreria standard 1164 della IEEE**
 - ▶ Tipologia non numerica che usa 9 valori

Valore	Significato
U	Non definito (a cui non è mai stato dato un valore)
X	Sconosciuto (il cui valore non è determinabile)
0	0 logico
1	1 logico
Z	Alta impedenza
W	Segnale debole, senza un valore determinabile
L	Segnale debole che probabilmente andrà a 0
H	Segnale debole che probabilmente andrà a 1
—	Indifferente (don't care)

Uso del tipo std_logic

► Sostituisce, estendendolo, il tipo bit

- Occorre però ridefinire tutti gli operatori (**and**, **or**, **xor**, etc.) per usufruire dei nuovi valori
- La ridefinizione si chiama *overloading* dell'operatore
- Per esempio l'operatore **and** è ri-definito come segue

	U	X	0	1	Z	W	L	H	–
U	U	U	0	U	U	U	0	U	U
X	U	X	0	X	X	X	0	X	X
0	0	0	0	0	0	0	0	0	0
1	U	X	0	1	X	X	0	1	X
Z	U	X	0	X	X	X	0	X	X
W	U	X	0	X	X	X	0	X	X
L	0	0	0	0	0	0	0	0	0
H	U	X	0	1	X	X	0	1	X
–	U	X	0	X	X	X	0	X	X



Uso di espressioni condizionate

- ▶ **Come esprimere delle condizioni in una equazione?**
 - ▶ In una espressione non possiamo usare lo statement **if**, abbiamo invece solo operatori
 - ▶ C'è bisogno di un operatore condizionato
- ▶ **E' presente per esempio in C o in Java**
 - ▶ `int z = (b < 10) ? a : a * b;`
- ▶ **In VHDL ve ne sono di due tipi**
 - ▶ Assegnazione condizionata
 - ▶ Assegnazione selezionata

Assegnazione condizionata

- ▶ Realizziamo, per esempio, un buffer tri-state
 - ▶ Se enable è attivo, l'uscita è uguale all'ingresso
 - ▶ Altrimenti è in alta impedenza 'Z'
- ▶ Direttiva **when ... else**
 - ▶ Funziona come l'if, ma in forma di equazione

```
-- Assegnazione condizionata, il buffer tri-state
library ieee;
use ieee.std_logic_1164.all;

entity three_state is
  port (a, enable : in std_logic;
        y : out std_logic);
end entity three_state;

architecture when_else of three_state is
begin
  y <= a when enable = '1' else 'Z';
end architecture when_else;
```

Inclusione della libreria standard 1164

if enable = 1 then y = a
else y = 'Z';

L'operatore di uguaglianza
è = invece di == come in
altri linguaggi

Scelte annidate

- ▶ La parte else può essere una qualunque espressione
 - ▶ In particolare può essere un altro **when ... else**
 - ▶ E' possibile quindi fare scelte multiple
- ▶ Realizziamo un decoder 2-4
 - ▶ Il simulatore analizza le varie condizioni nell'ordine in cui compaiono
 - ▶ La prima che risulta verificata viene presa, le rimanenti sono ignorate (quindi non devono essere necessariamente mutualmente esclusive)

```
-- Assegnazione condizionata, il decoder 2-4
library ieee;
use ieee.std_logic_1164.all;

entity decoder is
    port (a : in std_logic_vector(1 downto 0);
          y : out std_logic_vector(3 downto 0));
end entity decoder;

architecture when_else of decoder is
begin
    y <= "0001" when a = "00" else
        "0010" when a = "01" else
        "0100" when a = "10" else
        "1000" when a = "11" else
        "XXXX";
end architecture when_else;
```

Array di ingressi e uscite

Doppi apici quando si
definisce un valore multibit

Possibili alternative

Alternativa finale: il
segnale *a* potrebbe avere X
o altri valori non binari

Assegnazione selezionata

- ▶ E' come lo switch ... case di molti linguaggi di programmazione
 - ▶ Si scelgono le alternative sulla base del valore di un segnale
 - ▶ Le alternative vengono considerate contemporaneamente, devono quindi essere mutualmente esclusive

```
-- Assegnazione selezionata, il decoder 2-4
library ieee;
use ieee.std_logic_1164.all;

entity decoder is
  port (a : in std_logic_vector(1 downto 0);
        y : out std_logic_vector(3 downto 0));
end entity decoder;

architecture with_select of decoder is
begin
  with a select
    y <= "0001" when "00",
         "0010" when "01",
         "0100" when "10",
         "1000" when "11",
         "XXXX" when others;
end architecture with_select;
```

Segnale su cui eseguire la selezione

Valori dei vari casi.
Attenzione: sono valori,
non espressioni booleane!

Per tutti gli altri casi

Alternative equivalenti

- ▶ Quando diverse alternative danno lo stesso risultato si possono raggruppare
 - ▶ Realizziamo un decoder a 7 segmenti decimale
 - ▶ Le configurazioni oltre il 9 mostrano la lettera E per segnalare errore

```
-- Decoder 7 segmenti
entity seven_seg is
  port (a : in std_logic_vector(3 downto 0);
        y : out std_logic_vector(6 downto 0));
end entity seven_seg;

architecture with_select of seven_seg is
begin
  with a select
    y <= "0111111" when "0000",
         "0000110" when "0001",
         "1011011" when "0010",
         "1001111" when "0011",
         "1100110" when "0100",
         "1101101" when "0101",
         "1111101" when "0110",
         "0000111" when "0111",
         "1111111" when "1000",
         "1101111" when "1001",
         "1111001" when "1010" | "1011" | "1100" |
                                "1101" | "1110" | "1111",
         "0000000" when others;
end architecture with_select;
```



Alternative equivalenti
separate da |

Differenza semantica

- ▶ Usare uno o l'altro metodo è spesso lo stesso
 - ▶ Ma vi sono delle sostanziali differenze semantiche
 - ▶ L'assegnazione condizionata può guardare per esempio condizioni su più segnali diversi

```
-- Multiplexer, assegnazione selezionata
library ieee;
use ieee.std_logic_1164.all;

entity multiplexer is
    port (a, b, c : in std_logic;
          sa, sb, sc : in std_logic;
          y : out std_logic);
end entity multiplexer;

architecture with_select of multiplexer is
    signal selez : std_logic_vector(2 downto 0);
begin
    selez <= (sa, sb, sc);
    with selez select
        y <= a when "100",
            b when "010",
            c when "001",
            '0' when others;
end architecture with_select;
```

Concatenazione di
segnali

```
-- Multiplexer, assegnazione condizionata
library ieee;
use ieee.std_logic_1164.all;

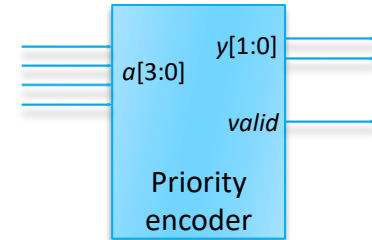
entity multiplexer is
    port (a, b, c : in std_logic;
          sa, sb, sc : in std_logic;
          y : out std_logic);
end entity multiplexer;

architecture when_else of multiplexer is
begin
    y <= a when sa = '1' else
        b when sb = '1' else
        c when sc = '1' else
        '0';
end architecture when_else;
```

Non mutualmente
esclusivi

Priority encoder e don't care

- ▶ I don't care in std_logic sono valori come altri
 - ▶ Non vengono interpretati come ci aspettiamo
 - ▶ Meglio usare condizioni nidificate con **when ... else**



```
-- Priority encoder, don't care (non funziona)
library IEEE;
use IEEE.std_logic_1164.all;

entity priority is
    port (a: in std_logic_vector(3 downto 0);
          y: out std_logic_vector(1 downto 0);
          valid: out std_logic);
end entity priority;

architecture DontCare of priority is
begin
    with a select
        y <= "00" when "0001",
            "01" when "001-",
            "10" when "01--",
            "11" when "1---",
            "00" when others;
    valid <= '1' when a(0) = '1' or a(1) = '1' or
                  a(2) = '1' or a(3) = '1'
            else '0';
end architecture DontCare;
```

```
-- Priority encoder, when ... else
library IEEE;
use IEEE.std_logic_1164.all;

entity priority is
    port (a: in std_logic_vector(3 downto 0);
          y: out std_logic_vector(1 downto 0);
          valid: out std_logic);
end entity priority;

architecture Ordered of priority is
begin
    y <= "11" when a(3) = '1' else
        "10" when a(2) = '1' else
        "01" when a(1) = '1' else
        "00" when a(0) = '1' else
        "00";
    valid <= '1' when a(0) = '1' or a(1) = '1' or
                  a(2) = '1' or a(3) = '1'
            else '0';
end architecture Ordered;
```

Stile comportamentale

I processi

Stile comportamentale

- ▶ **Prevedere molte alternative può diventare laborioso**
 - ▶ Ogni uscita deve avere una sola espressione
 - ▶ Diventa difficile riuscire a partizionare il calcolo
- ▶ **Per questo è possibile usare uno stile di specifica comportamentale**
 - ▶ Sono parti di codice eseguite in sequenza, invece che in concorrenza
 - ▶ Si realizzano all'interno di processi
 - ▶ Il codice è simile a quello che si scrive per i tradizionali linguaggi di programmazione
 - ▶ Utile sia per circuiti combinatori che per circuiti sequenziali
- ▶ **Fondamentale capire come e quando viene attivato il codice durante la simulazione**
 - ▶ Arricchisce la semantica vista fino ad ora

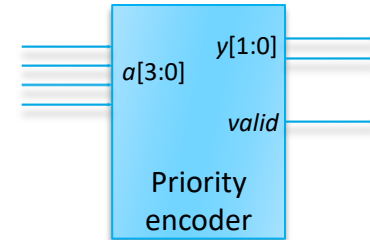
Processo

- ▶ **Si usa all'interno di una architettura**
 - ▶ Lo si usa al posto di una equazione
- ▶ **Sensitivity list**
 - ▶ Una lista di segnali che definiscono le condizioni di attivazione del processo
 - ▶ Il processo viene eseguito ogni volta che uno dei segnali nella sensitivity list cambia di valore

```
-- Sintassi di un processo
architecture comportamentale of mio_modulo is
begin
  label: process (sensitivity-list) is
    dichiarazioni di tipi, costanti, segnali, variabili;
  begin
    statement-sequenziale;
    ...
    statement-sequenziale;
  end process;
end architecture comportamentale;
```

Rivediamo il priority encoder

- ▶ Le uscite vanno aggiornate ogni volta che cambia l'ingresso *a*
 - ▶ Definiamo allora un processo sensibile alle variazioni di *a*
- ▶ Una serie di if annidati controllano gli ingressi attivi
 - ▶ Cominciamo con quello a priorità più alta
 - ▶ Un ultimo caso gestisce tutte le altre alternative
- ▶ Componente combinatorio
 - ▶ Anche se abbiamo usato uno stile comportamentale, l'effetto totale è combinatorio
 - ▶ Dato *a* è sempre possibile determinare il valore di tutte le uscite
 - ▶ Il processo è come una espressione che pilota il valore di *y* e di *valid*



```
-- Priority encoder, comportamentale
architecture sequential of priority is
begin
  process (a) is
  begin
    if a(3) = '1' then
      y <= "11";
      valid <= '1';
    elsif a(2) = '1' then
      y <= "10";
      valid <= '1';
    elsif a(1) = '1' then
      y <= "01";
      valid <= '1';
    elsif a(0) = '1' then
      y <= "00";
      valid <= '1';
    else
      y <= "00";
      valid <= '0';
    end if;
  end process;
end architecture sequential;
```

Semantica

▶ Attivazione

- ▶ Ogni volta che un segnale ha un evento il simulatore guarda quali processi sono sensibili verificando le sensitivity list
- ▶ Attiva quelli che hanno il segnale nella lista

▶ Esecuzione

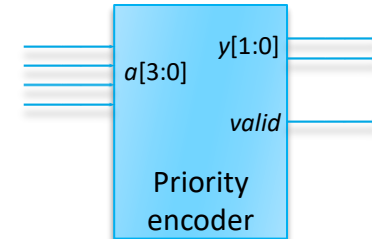
- ▶ Il simulatore esegue in sequenza gli statement contenuti nel processo
- ▶ Il tempo di simulazione NON avanza durante l'esecuzione
- ▶ I nuovi valori dei segnali NON vengono assegnati immediatamente
- ▶ L'assegnazione avviene invece solo dopo che il processo termina o viene sospeso (tipicamente quando si arriva al fondo del processo)

▶ Assegnazione dei segnali

- ▶ Uno stesso segnale (di uscita) può essere assegnato più volte in un processo, anche con valori diversi
- ▶ Tanto l'assegnazione non si fa subito, ma alla fine del processo
- ▶ Alla fine del processo si assegna effettivamente l'ultimo dei valori assegnati

Valori di default per circuiti combinatori

- ▶ Utile assegnare un valore a tutte le uscite ad inizio processo
 - ▶ Nel processo è poi necessario solamente gestire i casi non già coperti dal default
 - ▶ Questo inoltre garantisce contro certi errori (inferenza di latch) che vedremo in seguito
 - ▶ Nel caso del priority encoder, di default assumiamo che l'uscita sia valida e poniamo `valid <= '1'`;
 - ▶ Se `a = "0000"` allora poniamo `valid <= '0'`
 - ▶ Notate che `valid` viene assegnato due volte durante l'esecuzione quando `a = "0000"`



```
-- Priority encoder, comportamentale
architecture sequential of priority is
begin
  process (a) is
  begin
    valid <= '1';
    if a(3) = '1' then
      y <= "11";
    elsif a(2) = '1' then
      y <= "10";
    elsif a(1) = '1' then
      y <= "01";
    elsif a(0) = '1' then
      y <= "00";
    else
      y <= "00";
      valid <= '0';
    end if;
  end process;
end architecture sequential;
```

Variabili

- ▶ **Quando si scrivono algoritmi iterativi (loop), è utile aggiornare certe quantità in modo incrementale**
 - ▶ Questo **non lo si può fare con i segnali**, perché vengono aggiornati **solo a fine processo**, e non cambierebbero durante tutta l'esecuzione del loop
 - ▶ Nei processi è allora possibile definire delle **variabili**
 - ▶ **Le variabili prendono il nuovo valore immediatamente quando gli viene assegnato**
 - ▶ Quindi se vengono assegnate più volte durante il processo, il loro valore cambia di volta in volta (mentre per i segnali cambierebbe una sola volta, alla fine)
 - ▶ **Mantengono il valore tra una attivazione e la successiva**
- ▶ **Assegnazione**
 - ▶ L'operatore di assegnazione per le variabili è **:=**
 - ▶ Per distinguerlo da quello dei segnali

Controllo parità

► Realizziamo un controllo di parità in modo iterativo

- Assumiamo inizialmente che una parola sia pari
- Eseguiamo un ciclo **for** che analizza tutti i bit della parola di ingresso
- Ogni volta che si incontra un 1 aggiorniamo la parità
 - Se era pari diventa dispari, se era dispari diventa pari
- Quello che ci rimane alla fine è la parità effettiva

► Se la variabile fosse un segnale non funzionerebbe

- Il segnale non cambierebbe valore da una iterazione alla successiva

► Il modello è comunque combinatorio

- Anche se usa un ciclo, ricordate che il tempo non sta andando avanti
- Usiamo un segnale *y* per propagare il risultato all'esterno

```
-- Controllo di parità
library ieee;
use ieee.std_logic_1164.all;

entity parity is
  port (a : in std_logic_vector;
        y : out std_logic);
end entity parity;

architecture iterative of parity is
begin
  process (a) is
    variable even : std_logic;
  begin
    even := '0';
    for i in a'range loop
      if a(i) = '1' then
        even := not even;
      end if;
    end loop;
    y <= even;
  end process;
end architecture iterative;
```

No range

Definizione
variabile

Range desunto
dal contesto

Assegnamento
variabile

Risultato finale

Segnali e variabili

Segnali	Variabili
Usati per la comunicazione tra componenti e nello stile data flow	Usato solo per memorizzare dati
Può essere visto come un segnale reale di un circuito, essendo disponibile per essere condiviso tra più componenti	L'uso è locale al processo, non può essere condiviso tra componenti diversi
L'assegnazione avviene con un certo ritardo rispetto al tempo della sua esecuzione	L'assegnazione avviene immediatamente al tempo della sua esecuzione

Confronto tra i due usi

- ▶ **I risultati usando variabili e segnali sono differenti**
 - ▶ Supponiamo che i segnali x e y valgano inizialmente 1
 - ▶ Se y va a 0, entrambi i processi vengono attivati
 - ▶ Nell'esempio di sinistra, l'assegnamento a x viene deferito fino alla fine del processo, quindi z a fine processo prende il valore 0, perché x valeva ancora 1 al momento della valutazione
 - ▶ Nell'esempio di destra, x è una variabile, quindi il suo valore viene aggiornato immediatamente a 0; alla valutazione dell'espressione di z si ottiene quindi 1

```
-- Algoritmo con segnali
architecture ... is
    signal x : std_logic;
begin
    process (y) is
    begin
        x <= y;
        z <= not x;
    end process;
end architecture;

x = 0; y = 0; z = 0
```

```
-- Algoritmo con variabili
architecture ... is
begin
    process (y) is
        variable x : std_logic;
    begin
        x := y;
        z <= not x;
    end process;
end architecture;

x = 0; y = 0; z = 1
```

Equazioni e processi

	Equazioni	Processi
Attivazione	Ogni volta che un segnale a destra di $\leq$ cambia valore	Ogni volta che un segnale in sensitivity list cambia valore
Pilota	Determina il valore di un solo segnale	Può determinare il valore di più segnali contemporaneamente
Assegnazione	Evento inserito in coda dopo un delta cycle o il ritardo specificato	Evento/i inserito in coda a sospensione processo, dopo delta cycle o ritardo specificato
Conflitti	Altre equazioni o processi non possono pilotare lo stesso segnale	Altre equazioni o processi non possono pilotare gli stessi segnali
Variabili	No	Sì

Testbench con processi

▶ I processi semplificano la scrittura dei testbench

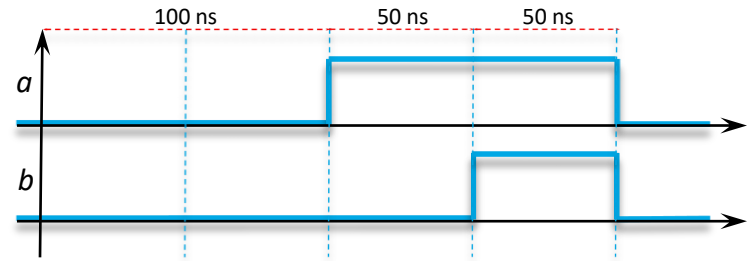
- ▶ Usiamo un processo senza sensitivity list per definire i segnali a priori
- ▶ Poiché non c'è sensitivity list, il processo viene attivato immediatamente all'inizio della simulazione
- ▶ Appena termina viene riattivato immediatamente
- ▶ Allora come lo fermo?

▶ Il comando wait

- ▶ Se incontriamo un comando **wait** all'interno di un processo l'esecuzione si interrompe per un tempo specificato dal comando
- ▶ Oppure si può interrompere il processo per sempre se non si specifica il tempo
- ▶ Quando si interrompe il processo, si eseguono gli assegnamenti ai segnali (come se terminasse)

Uso del wait

- ▶ Si specifica un tempo di sospensione con **wait for**
 - ▶ Il processo verrà riattivato quando passa il tempo specificato dal punto in cui era stato sospeso
 - ▶ Questo ha il vantaggio di poter usare tempi relativi invece che assoluti come fatto precedentemente
- ▶ Sospensione completa
 - ▶ Si usa **wait** e basta per terminare il processo



```
-- Testbench per porta And a 2 ingressi
entity TestAnd2 is
end entity TestAnd2;

architecture simple of TestAnd2 is
    -- Segnali interni di interconnessione
    signal a, b, c : bit;
begin
    -- Istanza del modulo da testare
    g1: And2 port map (x => a, y => b, z => c);
    -- Definizione degli stimoli
    a <= '0', '1' after 100 ns, '0' after 200 ns;
    b <= '0', '1' after 150 ns, '0' after 200 ns;
end architecture simple;
```

```
architecture simple of TestAnd2 is
    -- Segnali interni di interconnessione
    signal a, b, c : bit;
begin
    -- Istanza del modulo da testare
    g1: And2 port map (x => a, y => b, z => c);
    -- Definizione degli stimoli
    process is
    begin
        a <= '0';
        b <= '0';
        wait for 100 ns;
        a <= '1';
        wait for 50 ns;
        b <= '1';
        wait for 50 ns;
        a <= '0';
        b <= '0';
        wait;
    end process;
end architecture simple;
```